# Quiz — 2.3.1 Algorithms — ANSWER SHEET

*Separate answer sheet / mark scheme — keep for marking.*

## Answer key

### Section A (10 marks — 1 each)

| Q | Ans | Why |
|---|---|---|
| A1 | **B** | Binary search halves the search space each step → O(log n) worst case. |
| A2 | **B** | Merge sort divides into log n levels, each doing n work to merge → O(n log n) in all cases. |
| A3 | **C** | Quick sort degrades to O($n^2$) with a poor pivot (e.g. already-sorted data); average is O(n log n). |
| A4 | **A** | Binary search requires a **sorted** list so a half can be safely discarded each pass. |
| A5 | **B** | BFS uses a **queue** (FIFO) to process nodes level by level. (DFS uses a stack.) |
| A6 | **C** | Drop constants and lower-order terms; the dominant term $n^2$ gives O($n^2$). |
| A7 | **A** | Stack push/pop/peek are O(1) — they act only on the top, no traversal. |
| A8 | **B** | In-order (Left → Node → Right) of a BST yields values in ascending order. |
| A9 | **B** | Only solvable in exponential time or worse = **intractable**. |
| A10 | **B** | A* adds an admissible heuristic h(n) estimating remaining cost to the goal (f = g + h), directing the search. |

### Section B (20 marks)

**B1 [3]** — **O($n^2$)**. *(1 mark)* Justification *(2 marks)*: There is a loop nested inside a loop. The outer loop runs **n** times, and for each outer iteration the inner loop runs **n** times, giving roughly **n × n = $n^2$** comparisons. As constants and lower-order terms are dropped, this is **O($n^2$)** (quadratic) — doubling n quadruples the work.

**B2 [4]** — Bubble sort passes (largest value bubbles to the right each pass). Award 1 mark per correct pass row.

Start: `[5, 3, 8, 1, 4]`

| After pass | Result |
|---|---|
| Pass 1 | `[3, 5, 1, 4, 8]` |
| Pass 2 | `[3, 1, 4, 5, 8]` |
| Pass 3 | `[1, 3, 4, 5, 8]` |
| Pass 4 | `[1, 3, 4, 5, 8]` (no swaps — sorted) |

Working for Pass 1: compare 5,3→swap `[3,5,8,1,4]`; 5,8→no swap; 8,1→swap `[3,5,1,8,4]`; 8,4→swap `[3,5,1,4,8]`. Pass 2: 3,5→no; 5,1→swap `[3,1,5,4,8]`; 5,4→swap `[3,1,4,5,8]`; 5,8→no. Pass 3: 3,1→swap `[1,3,4,5,8]`; 3,4→no; 4,5→no; 5,8→no. Pass 4: all in order, no swaps.

**B3 [4]** — Binary search for **23** in `[4, 9, 15, 18, 23, 27, 31, 42, 56]`. Award up to 4 for a correct trace.

| Step | low | high | mid (index) | value at mid | vs target 23 |
|---|---|---|---|---|---|
| 1 | 0 | 8 | 4 | 23 | = → **found** |

Note: mid = (0 + 8) DIV 2 = 4; value at index 4 is 23, which equals the target, so the search succeeds on the **first** step. Result: **23 found at index 4** (1 mark). (Full marks for showing low=0, high=8, mid=4, identifying value 23, and concluding "found at index 4". If a candidate's list/target leads to multiple steps, the same method — recompute mid, discard a half by adjusting low/high — earns the marks.)

**B4 [3]** — Tree: root 40; left subtree 25 (children 15, 30); right subtree 60 (children 50, 70).

(a) **Pre-order** (Node → Left → Right): **40, 25, 15, 30, 60, 50, 70** *(1)* (b) **In-order** (Left → Node → Right): **15, 25, 30, 40, 50, 60, 70** *(1)* (c) **Post-order** (Left → Right → Node): **15, 30, 25, 50, 70, 60, 40** *(1)*

**B5 [3]** — - **Depth-first traversal (DFS)** uses a **stack** (or recursion, which uses the call stack). *(1)* - **Breadth-first traversal (BFS)** uses a **queue**. *(1)* - One use of BFS: finding the **shortest path in an unweighted graph** (also accept: web crawling, social-network "degrees of separation", peer-to-peer searching). *(1)*

**B6 [3]** — Dijkstra from A. Final shortest distances:

| Node | Shortest distance from A | Via |
|---|---|---|
| A | 0 | — |
| D | 1 | A |
| E | 3 | D |
| F | 4 | E |
| B | 4 | A |
| C | 6 | B |

Working: - A = 0 (start). - A–D = 1, A–B = 4. Settle **D = 1** (smallest). - From D: E = 1 + 2 = **3**. Settle **E = 3**. - From E: F = 3 + 1 = **4**; B via E = 3 + 5 = 8 (worse than 4, keep B = 4). Settle **B = 4** and **F = 4** (both 4; either order). - From B: C = 4 + 2 = **6**. From F: C via F = 4 + 3 = 7 (worse, keep 6). Settle **C = 6**.

Award up to 3 marks: B=4, C=6, D=1, E=3, F=4 (e.g. 1 mark per 2 correct nodes, rounded in candidate's favour). Common correct alternative "via" for C is B; for F is E.

## Section C (5 marks)

**C1 [5]** — Indicative content; award marks for valid comparison points and a justified recommendation. Mark band 4–5 for a developed comparison that explicitly justifies the choice using the single-target nature of the task.

- Both algorithms are **guaranteed to find the optimal (shortest) path** when edge weights are non-negative (A *requires an admissible heuristic that never overestimates).* (1)*
- **Dijkstra** uses only the actual cost so far, g(n), and searches **outwards in all directions**, finding the shortest path to **all** nodes. This explores **more nodes than necessary** when only one target is needed. *(1)*

- **A*** uses **f(n) = g(n) + h(n)**, adding a heuristic estimate h(n) of the remaining distance to the goal (e.g. straight-line/Manhattan distance). This **directs the search towards the target**. *(1)*
- Because the target is a **single, known location**, A *typically explores far fewer nodes and so is faster / more efficient, which matters on a large map in a real-time game where performance is critical.* (1)*
- **Recommendation: A*** is the better choice here, as it gives the same optimal route as Dijkstra but reaches a single known goal more efficiently. (Dijkstra would be preferable if shortest paths to *many* destinations were required, or if no good heuristic were available.) *(1)*

---

**Total: 35 marks** (Section A = 10, Section B = 3 + 4 + 4 + 3 + 3 + 3 = 20, Section C = 5).